浙江大学实验报告

姓名：[redacted] 专业：信息工程 学号：[redacted]

课程名称：信息与电子工程导论 任课老师：[redacted]

实验名称：基于 MATLAB 的信号频谱分析 实验日期：

1 实验目的和要求

1.1. 实验目的

(1) 构建一个信号，至少含有 3 个频率分量，可以是周期信号，也可以是非周期信号（如加随机噪声）；

(2) 利用 MATLAB 分析其频谱，并对频谱作适当的处理（滤波，或增加一些频率成分），观察信号的变化；

1.2. 实验要求

(1) 构建的信号至少含有 3 个频率分量（可以是周期信号，也可以是非周期信号）；

(2) 利用 MATLAB 分析其频谱；

(3) 利用 MATLAB 对其频谱做适当的处理，观察信号的变化；

2 实验原理

计算机可以处理离散的和有限长度的数值信号，并通过平滑连接模拟连续信号，同时，借助 MATLAB 等工具，还可以实现利用快速傅里叶变换和快速傅里叶逆变换实现信号时域与频域的互相转化，并能对信号进行一定的处理与改变。

3 实验内容

(1) 构建一个含有 3 个频率分量的周期信号；

(2) 用 MATLAB 对其进行傅里叶变换得到其频谱；

(3) 用 MATLAB 对(2)中得到频谱进行傅里叶逆变换得到其时域；

(4) 对(1)中构建的周期信号加入随机噪声；

(5) 用 MATLAB 对其进行傅里叶变换得到频谱；

(6) 对(5)中得到的频谱进行滤波处理，得到降噪过后的信号频谱，观察降噪效果；

(7) 用 MATLAB 对(6)中得到频谱进行傅里叶逆变换得到其时域，观察其波形；

4 实验结果和分析

(1) 构建一个含有 3 个频率分量的周期信号，并用 MATLAB 对其进行傅里叶变换得到其频谱，用 MATLAB 对(2)中得到频谱进行傅里叶逆变换得到其时域，并与初始时域进比较；

构建的信号如下：

$$x = 2.4sin(100\pi t) + 1.8sin(240\pi t) + 1.2sin(500\pi t);$$

使用的 MATLAB 代码如下：

```
Fs = 1000;
N  = 1000;
t  = (0:N-1)/Fs;
x = 2.4*sin(2*pi*50*t) + 1.8*sin(2*pi*120*t) + 1.2*sin(2*pi*250*t);
freqs = [50, 120, 250];
amps  = [2.4, 1.8, 1.2];

X = fft(x);
X_single = X(1:N/2+1);
f_single = (0:N/2) * (Fs/N);
magnitude_single = abs(X_single) / N;
magnitude_single(2:end-1) = 2 * magnitude_single(2:end-1);
figure;

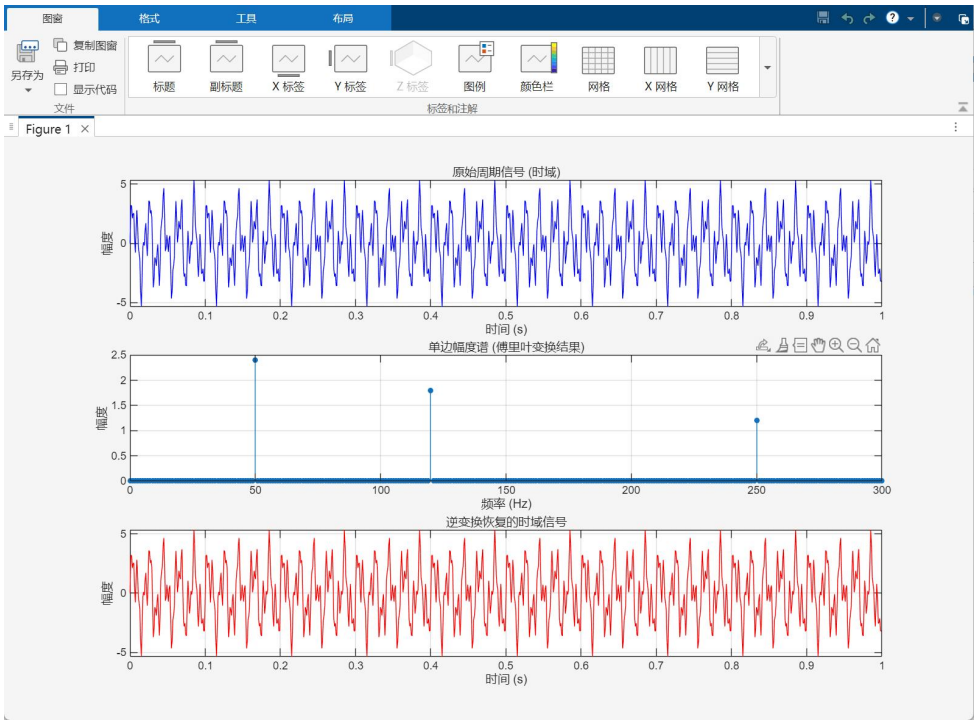
subplot(3,1,1);
plot(t, x, 'b');
xlabel('时间 (s)');
ylabel('幅度');
title('原始周期信号 (时域)');
grid on;

subplot(3,1,2);
stem(f_single, magnitude_single, 'filled', 'MarkerSize', 4);
xlabel('频率 (Hz)');
ylabel('幅度');
title('单边幅度谱 (傅里叶变换结果)');
xlim([0, 300]);
grid on;
hold on;
for i = 1:length(freqs)
    [~, idx] = min(abs(f_single - freqs(i)));
    plot(f_single(idx),  magnitude_single(idx),  'ro',  'MarkerSize',  8,
'LineWidth', 2);
    text(f_single(idx)+10, magnitude_single(idx)+0.05, ...
        sprintf('%.1f Hz, %.2f', f_single(idx), magnitude_single(idx)), ...
        'Color', 'r', 'FontSize', 10);
end
hold off;

x_recovered = ifft(X);
subplot(3,1,3);
plot(t, x_recovered, 'r');
xlabel('时间 (s)');
ylabel('幅度');
```

```
title('逆变换恢复的时域信号');
grid on;
```

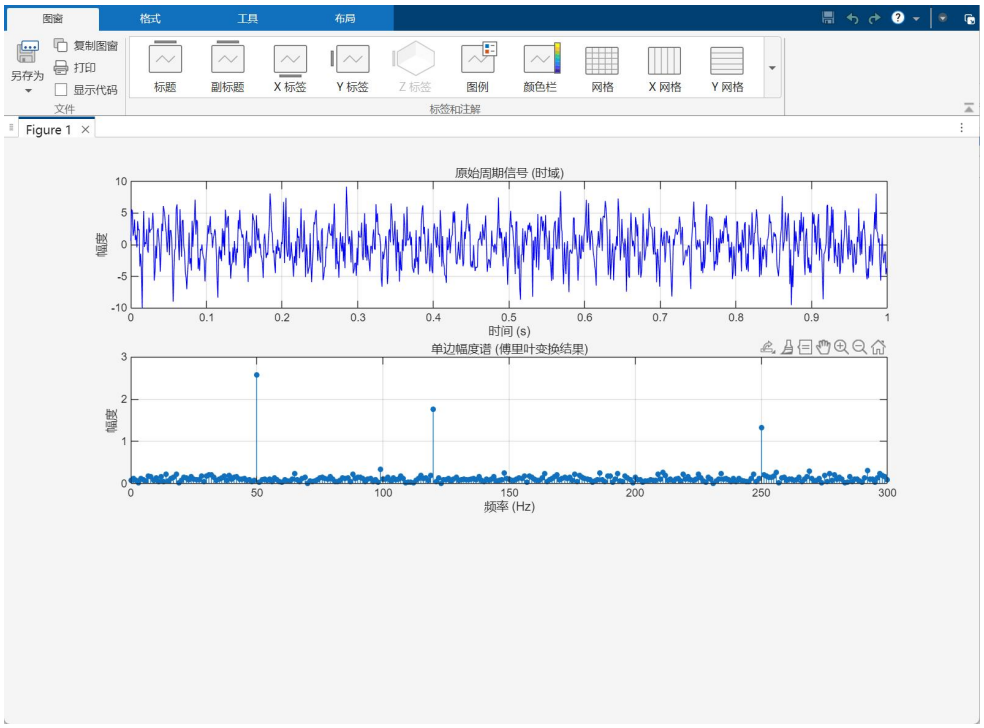
MATLAB 给出的图像如下：



(2) 对(1)中构建的周期信号加入随机噪声，并用 MATLAB 对其进行傅里叶变换得到频谱；将 MATLAB 的代码当中将 x 修改为

$$x = 2.4sin(100\pi t) + 1.8sin(240\pi t) + 1.2sin(500\pi t) + 2randn(size(t));$$

重复运行该代码，得到如下 figure：



(3) 对频谱进行滤波处理，得到去除噪声的信号的频谱，观察去除效果，并用 MATLAB 对该频谱进行傅里叶逆变换得到其时域，观察其波形；

结合(2)所给出的频域图，频率峰分别为 50Hz、120Hz、250Hz，因此，保留频率峰的波形，而舍去没有明显频率峰的频段，可得到无躁信号（后文会对其可操作性加以论述）；

使用如下的代码，将幅度较低的杂波成分滤去：

```
Fs = 1000;
N  = 1000;
t  = (0:N-1)/Fs;
x  =  2.4*sin(2*pi*50*t)  +  1.8*sin(2*pi*120*t)  +  1.2*sin(2*pi*250*t)  +
2*randn(size(t));

X = fft(x);
f_single = (0:N/2) * (Fs/N);
target_freqs = [50, 120, 250];

mask = zeros(1, N);
for f0 = target_freqs
    idx = round(f0 * N / Fs) + 1;
    if idx >= 1 && idx <= N/2+1
        mask(idx) = 1;
        mask(N - idx + 2) = 1;
    end
end

X_filtered = X .* mask;
x_filtered = ifft(X_filtered, 'symmetric');

X_filtered_single = X_filtered(1:N/2+1);
mag_filtered = abs(X_filtered_single) / N;
mag_filtered(2:end-1) = 2 * mag_filtered(2:end-1);

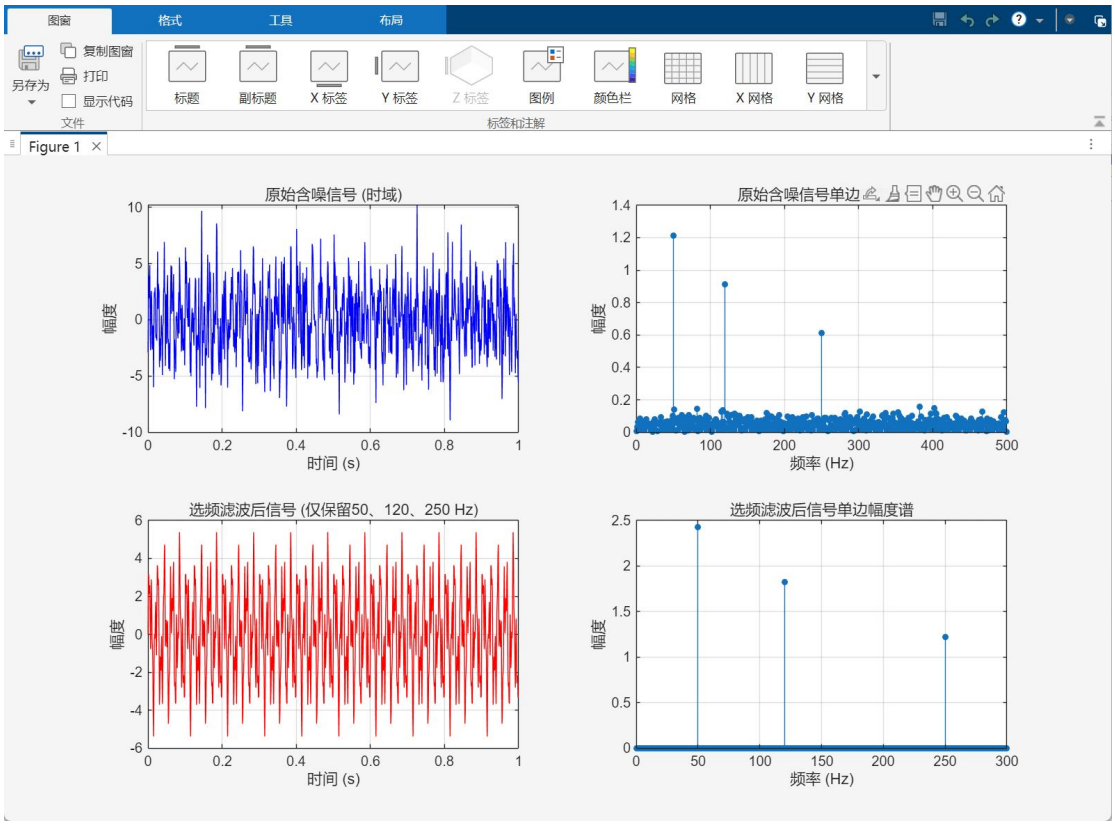
figure;
subplot(2,2,1);
plot(t, x, 'b');
xlabel('时间 (s)'); ylabel('幅度');
title('原始含噪信号 (时域)'); grid on;

subplot(2,2,2);
stem(f_single, abs(X(1:N/2+1))/N, 'filled', 'MarkerSize', 4);
xlabel('频率 (Hz)'); ylabel('幅度');
title('原始含噪信号单边幅度谱');
xlim([0, 500]); grid on;
```

```
subplot(2,2,3);
plot(t, x_filtered, 'r');
xlabel('时间 (s)'); ylabel('幅度');
title('选频滤波后信号 (仅保留 50、120、250 Hz)');
grid on;

subplot(2,2,4);
stem(f_single, mag_filtered, 'filled', 'MarkerSize', 4);
xlabel('频率 (Hz)'); ylabel('幅度');
title('选频滤波后信号单边幅度谱');
xlim([0, 300]); grid on;
```

MATLAB 得到的 figure 如下：



观察得到，其频谱与波形和原周期信号的波形一致；

5 实验结论

对于较为简单的周期信号进行处理时，可以优先考虑使用快速傅里叶变换，这样不仅能得到较为精确的结果，相比标准傅里叶变换还可以大幅度减少运算的步骤与耗时，从而减弱对硬件的需求，提高实验的效率与可操作性。对于受到随机信号干扰的信号，如果已知无噪信号幅度>>噪声信号的幅度（信噪比较高），可以直接选取频率峰的信号而过滤非频率峰的信号得到原信号；但大部分加噪信号部分频段可能与噪声信号的幅度差别不大，这样会使得一旦选择仅保留明显频率峰进行降噪处理后得到的信号可能会丢失部分低幅度频段，导致信号的失真。

6　源代码与分析

源代码及其注释如下。

无噪信号及其处理：

```
%% 参数设置
Fs = 1000;
N  = 1000;
t  = (0:N-1)/Fs;
%% 构建原始周期信号（直接写出三个频率分量的叠加）
% 三个频率分量：50 Hz, 120 Hz, 250 Hz，幅度分别为 2.4, 1.8, 1.2
x = 2.4*sin(2*pi*50*t) + 1.8*sin(2*pi*120*t) + 1.2*sin(2*pi*250*t);
%% 傅里叶变换并计算单边频谱，绘制
X = fft(x);
X_single = X(1:N/2+1);
f_single = (0:N/2) * (Fs/N);
magnitude_single = abs(X_single) / N;
magnitude_single(2:end-1) = 2 * magnitude_single(2:end-1);
figure;
% 子图 1：原始信号时域波形
subplot(3,1,1);
plot(t, x, 'b');
xlabel('时间 (s)');
ylabel('幅度');
title('原始周期信号 (时域)');
grid on;
% 子图 2：单边幅度谱
subplot(3,1,2);
stem(f_single, magnitude_single, 'filled', 'MarkerSize', 4);
xlabel('频率 (Hz)');
ylabel('幅度');
title('单边幅度谱 (傅里叶变换结果)');
xlim([0, 300]);
grid on;
%% 对频谱进行傅里叶逆变换恢复时域信号
x_recovered = ifft(X);
% 子图 3：逆变换恢复的时域信号
subplot(3,1,3);
plot(t, x_recovered, 'r');
xlabel('时间 (s)');
ylabel('幅度');
title('逆变换恢复的时域信号');
grid on;
```

加噪信号及其处理：

```
%% 参数设置
```

```
Fs = 1000;
N  = 1000;
t  = (0:N-1)/Fs;
%% 构建含噪信号
x = 2.4*sin(2*pi*50*t) + 1.8*sin(2*pi*120*t) + 1.2*sin(2*pi*250*t) +
2*randn(size(t));
%% 傅里叶变换
X = fft(x);
%% 频域理想选频滤波
f_single = (0:N/2) * (Fs/N);
f_double = (0:N-1) * (Fs/N);
% 目标频率
target_freqs = [50, 120, 250];
mask = zeros(1, N);
for f0 = target_freqs
    idx = round(f0 * N / Fs) + 1;
    if idx >= 1 && idx <= N/2+1
        mask(idx) = 1;
        mask(N - idx + 2) = 1;
    end
end
% 应用滤波器
X_filtered = X .* mask;
%% 逆变换得到滤波后时域信号
x_filtered = ifft(X_filtered, 'symmetric');   % 'symmetric' 确保输出为实信
号
X_filtered_single = X_filtered(1:N/2+1);
mag_filtered = abs(X_filtered_single) / N;
mag_filtered(2:end-1) = 2 * mag_filtered(2:end-1);
figure;
% 原始含噪信号
subplot(2,2,1);
plot(t, x, 'b');
xlabel('时间 (s)'); ylabel('幅度');
title('原始含噪信号 (时域)');
grid on;
% 原始含噪信号频谱
subplot(2,2,2);
stem(f_single, abs(X(1:N/2+1))/N, 'filled', 'MarkerSize', 4);
xlabel('频率 (Hz)'); ylabel('幅度');
title('原始含噪信号单边幅度谱');
xlim([0, 300]); grid on;
% 滤波后信号
subplot(2,2,3);
```

```
plot(t, x_filtered, 'r');
xlabel('时间 (s)'); ylabel('幅度');
title('选频滤波后信号 (仅保留 50、120、250 Hz)');
grid on;
% 滤波后信号频谱
subplot(2,2,4);
stem(f_single, mag_filtered, 'filled', 'MarkerSize', 4);
xlabel('频率 (Hz)'); ylabel('幅度');
title('选频滤波后信号单边幅度谱');
xlim([0,300]);
grid on;
```